CHAPTER 6

Device MANAGEMENT

4.1. I/O Hardware

- Computers operate a great many kinds of devices. General types include storage devices (disks, tapes), transmission devices (network cards, modems), and human-interface devices (screen, keyboard, mouse).
- A device communicates with a computer system by sending signals over a cable or even through the air. The device communicates with the machine via a connection point termed a port (for example, a serial port). If one or more devices use a common set of wires, the connection is called a bus.
- When device A has a cable that plugs into device B, and device B has a cable that plugs into device C, and device C plugs into a port on the computer, this arrangement is called a *daisy chain*. It usually operates as a bus.
- A controller is a collection of electronics that can operate a port, a bus, or a device. A serial-port controller is an example of a simple device controller. It is a single chip in the computer that controls the signals on the wires of a serial port.
- The SCSI bus controller is often implemented as a separate circuit board (a host adapter) that plugs into the computer. It typically contains a processor, microcode, and some private memory to enable it to process the SCSI protocol messages. Some devices have their own built-in controllers.
- An I/O port typically consists of four registers, called the *status*, *control*, *data-in*, and *data-out* registers. The status register contains bits that can be read by the host. These bits indicate states such as whether the current command has completed, whether a byte is available to be read from the data-in register, and whether there has been a device error. The control register can be written by the host to start a command or to change the mode of a device. For instance, a certain bit in the control register of a serial port chooses between full-duplex and half-duplex communication, another enables parity checking, a third bit sets the word length to 7 or 8 bits, and other bits select one of the speeds supported by the serial port.
- The data-in register is read by the host to get input, and the data out register is written by the host to send output. The data registers are typically 1 to 4 bytes. Some controllers have FIFO chips that can hold several bytes of input or output data to expand the capacity of the controller beyond the size of the data register. A FIFO chip can hold a small burst of data until the device or host is able to receive those data.

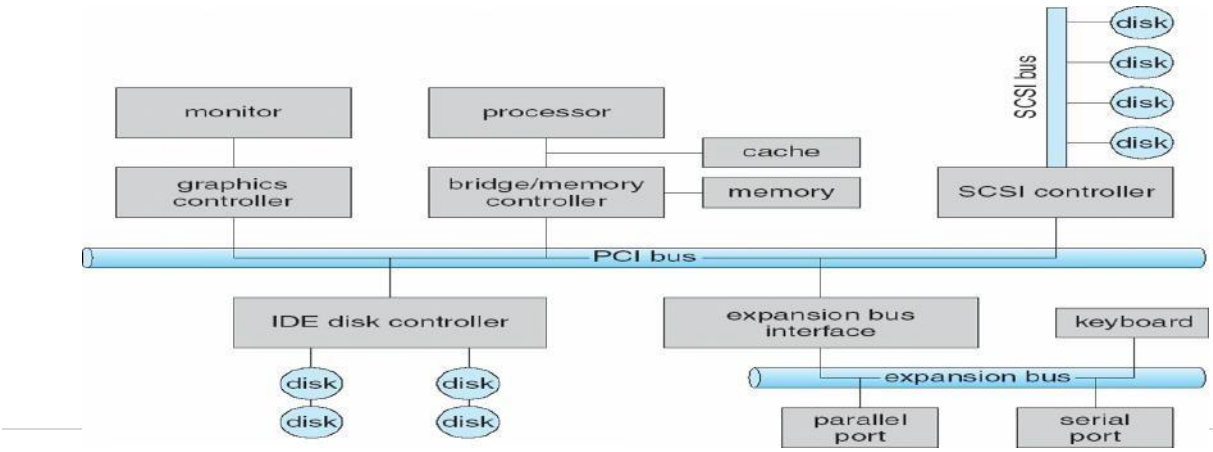


Figure 92: A typical PC bus structure

How can the processor give commands and data to a controller to accomplish an I/O transfer?

- Direct I/O instructions and Memory-mapped I/O

4.1.1. Direct I/O instructions

- The controller has one or more registers for data and control signals. The processor communicates with the controller by reading and writing bit patterns in these registers. One way in which this communication can occur is through the use of special I/O instructions that specify the transfer of a byte or word to an I/O port address. The I/O instruction triggers bus lines to select the proper device and to move bits into or out of a device register.

4.1.2. Memory-mapped I/O

- The device-control registers are mapped into the address space of the processor. The CPU executes I/O requests using the standard data-transfer instructions to read and write the device-control registers. Especially for large address spaces (graphics)

4.2. POLLING

- Incomplete protocol for interaction between the host and a controller can be intricate, but the basic handshaking notion is simple. The controller indicates its state through the busy bit in the status register. (Recall that to set a bit means to write a 1 into the bit, and to clear a bit mean to write a 0 into it.)
- The controller sets the busy bit when it is busy working, and clears the busy bit when it is ready to accept the next command. The host signals its wishes via the command-ready bit in the command register. The host sets the command-ready bit when a command is available for the controller to execute.
- For this example, the host writes output through a port, coordinating with the controller by handshaking as follows.
  1. The host repeatedly reads the busy bit until that bit becomes clear.
  2. The host sets the write bit in the command register and writes a byte into the data-out register.
  3. The host sets the command-ready bit.
  4. When the controller notices that the command-ready bit is set, it sets the Busy.
  5. The controller reads the command register and sees the write command.
  6. It reads the data-out register to get the byte, and does the I/O to the device.
  7. The controller clears the command-ready bit, clears the error bit in the status register to indicate that the device I/O succeeded, and clears the busy bit to indicate that it is finished.

- The host is busy-waiting or polling: It is in a loop, reading the status register over and over until the busy bit becomes clear. If the controller and device are fast, this method is a reasonable one. But if the wait may be long, the host should probably switch to another task

4.3. Interrupts

The basic interrupt mechanism works as follows.

- The CPU hardware has a wire called the interrupt request line that the CPU senses after executing every instruction.
- When the CPU detects that a controller has asserted a signal on the interrupt request line, the CPU performs a state save and jumps to the interrupt handler routine at a fixed address in memory.
- The interrupt handler determines the cause of the interrupt, performs the necessary processing, performs a state restore, and executes a return from interrupt instruction to return the CPU to the execution state prior to the interrupt.
- **In Short**: The device controller raises an interrupt by asserting a signal on the interrupt request line, the CPU catches the interrupt and dispatches it to the interrupt handler, and the handler clears the interrupt by servicing the device.

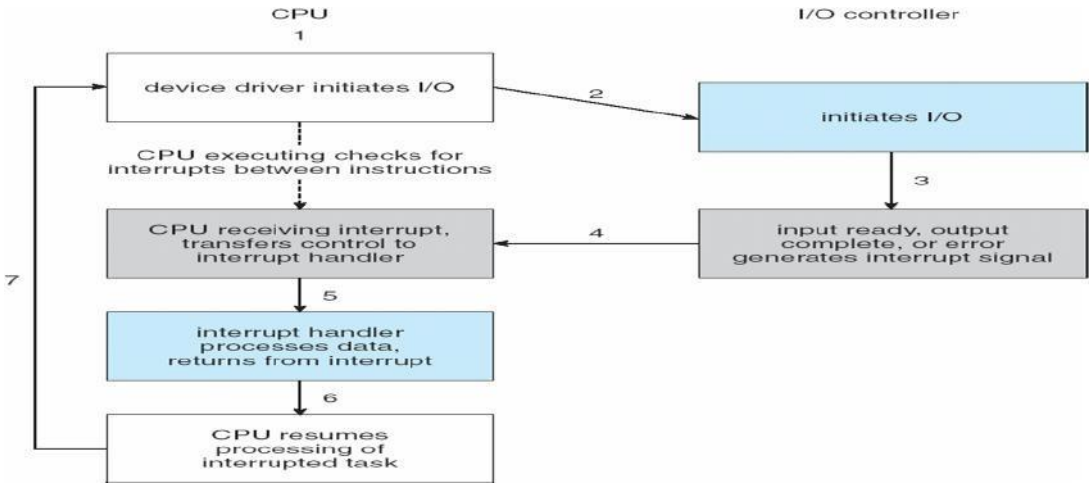


Figure 93: Interrupts driven I/O Cycle

- In a modern OS, we need more sophisticated interrupt handling features.
    1. We need the ability to defer interrupt handling during critical processing.
    2. We need an efficient way to dispatch to the proper interrupt handler for a device without first polling all the devices to see which one raised the interrupt.
    3. We need multilevel interrupts, so that the operating system can distinguish between high- and low-priority interrupts and can respond with the appropriate degree of urgency.
- In modern computer hardware, these three features are provided by the CPU and by the Interrupt-Controller hardware.
- Most CPUs have two interrupt request lines.
- One is the *non-maskable*, which is reserved for events such as unrecoverable memory errors.
- The second interrupt line is *maskable*, it can be turned off by the CPU before the execution of critical instruction sequences that must not be interrupted.
- The *maskable* interrupt is used by device controllers to request service.
- The interrupt mechanism accepts an address, this address is an offset in a table called the Interrupt Vector Table (IVT).
- This vector contains the memory addresses of specialized interrupt handlers.
- The interrupt mechanism also implements a system of Interrupt Priority Level.
- This mechanism enables the CPU to defer the handling of low-priority interrupts without masking off all interrupts and makes it possible for a high-priority interrupt to preempt the execution of a low-priority interrupt.

- A modern OS interacts with the interrupt mechanism in several ways.
    - *At boot time*: the OS probes the hardware buses to determine what devices are present and installs the corresponding interrupt handlers into the interrupt vector.
    - *During I/O*: the various device controllers raise interrupts when they are ready for service.
- These interrupts signify that output has completed, or that input data are available, or that a failure has been detected.
- Interrupt mechanism also used for **exceptions.**
- Terminate process, crash system due to hardware error, accessing a protected memory address or attempting to execute privilege instruction from user mode, divide by zero etc.
- Page fault executes when memory access error.
- System call executes via **trap** to trigger kernel to execute request.
- Multi-CPU systems can process interrupts concurrently.
- If operating system designed to handle it.
- Used for time-sensitive processing, frequent, must be fast.

4.4. I/O Devices

Categories of I/O Devices

1. Human readable
2. machine readable
3. Communication

1. Human Readable is suitable for communicating with the computer user.
    - Examples are printers, video display terminals, keyboard etc.
2. Machine Readable is suitable for communicating with electronic equipment.
    - Examples are disk and tape drives, sensors, controllers and actuators.3. Communication is suitable for communicating with remote devices.
    - Examples are digital line drivers and modems.
    - Differences between I/O Devices
        i. Data rate: there may be differences of several orders of magnitude between the data transfer rates.
        ii. Application: Different devices have different use in the system.
        iii. Complexity of Control: A disk is much more complex whereas printer requires simple control interface.
        iv. Unit of transfer: Data may be transferred as a stream of bytes or characters or in larger blocks.
        v. Data representation: Different data encoding schemes are used for different devices.
        vi. Error Conditions: The nature of errors differs widely from one device to another.

4.5. Ways to INPUT/OUTPUT

- There are three fundamentally different ways to do I/O
    1. Programmed I/O
    2. Interrupt-driven
    3. Direct Memory access

4.5.1. Programmed I/O

The processor issues an I/O command, on behalf of a process, to an I/O module; that process then busy waits for the operation to be completed before proceeding. When the processor is executing a program and encounters an instruction relating to input/output, it executes that instruction by issuing a command to the appropriate input/output module. With the programmed input/output, the input/output module will perform the required action and then set the appropriate bits in the input/output status register. The input/output module takes no further action to alert the processor. In particular it doesn't interrupt the processor. Thus, it is the responsibility of the processor to check the status of the input/output module periodically, until it finds that the operation is complete.
It is simplest to illustrate programmed I/O by means of an example. Consider a process that wants to print the eight character string ABCDEFGH.

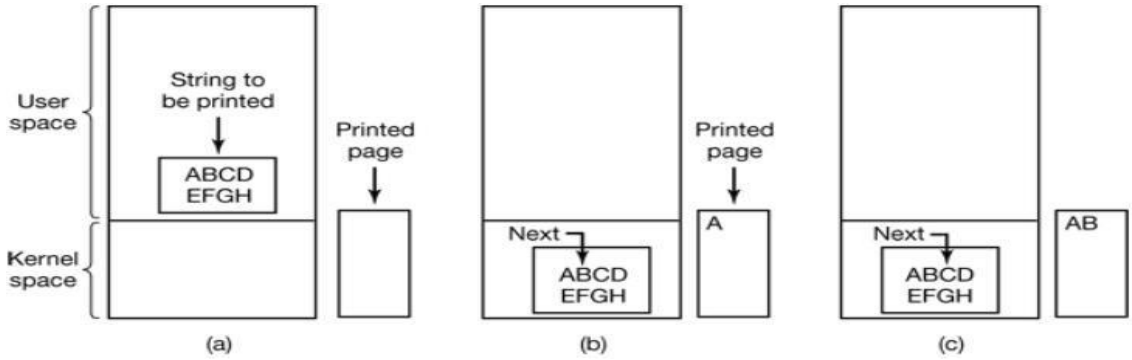


Figure 94: Steps in printing a string

1. It first assemble the string in a buffer in user space as shown in fig.
2. The user process then acquires the printer for writing by making system call to open it.
3. If printer is in use by other the call will fail and enter an error code or will block until printer is available, depending on OS and the parameters of the call.
4. Once it has printer the user process makes a system call to print it.
5. OS then usually copies the buffer with the string to an array, say P in the kernel space where it is more easily accessed since the kernel may have to change the memory map to get to user space.
6. As the printer is available the OS copies the first character to the printer data register, in this example using memory mapped I/O. This action activates the printer. The character may not appear yet because some printers buffer a line or a page before printing.
7. As soon as it has copied the first character to the printer the OS checks to see if the printer is ready to accept another one.
8. Generally printer has a second register which gives its status

The action followed by the OS are summarized in fig below. First data are copied to the kernel, then the OS enters a tight loop outputting the characters one at a time. The essentials aspects of programmed I/O is after outputting a character, the CPU continuously polls the device to see if it is ready to accept one.
This behavior is often called polling or Busy waiting.

```
copy_from_user(buffer,p,count); /*P is the kernel buffer*/
for(i=0;i<count;i++) {
 /* loop on every characters*/
```

```
while (*printer_status_reg!=READY); /*loop until ready*/
printer_data_register=P[i]; /*output one character */
}
return_to_user();
```

Programmed I/O is simple but has disadvantages of tying up the CPU full time until all the I/O is done. In an embedded system where the CPU has nothing else to do, busy waiting is reasonable. However in more complex system where the cpu has to do other things, busy waiting is inefficient. A better I/O method is needed.

4.5.2. Interrupt driven I/O

The problem with the programmed I/O is that the processor has to wait a long time for the input/output module of concern to be ready for either reception or transmission of more data. The processor, while waiting, must repeatedly interrogate the status of the Input/ Output module. As a result the level of performance of entire system is degraded.

An alternative approach for this is interrupt driven Input/Output. The processor issue an Input/output command to a module and then go on to do some other useful work. The input/ Output module will then interrupt the processor to request service, when it is ready to exchange data with the processor. The processor then executes the data transfer as before and then resumes its former processing. Interrupt-driven input/output still consumes a lot of time because every data has to pass with processor.

4.5.3. Direct Memory Access (DMA)

- A special control unit may be provided to allow transfer of a block of data directly between an external device and the main memory, without continuous intervention by the processor. This approach is called ***Direct Memory Access (DMA).***
- DMA can be used with either polling or interrupt software. DMA is particularly useful on devices like disks, where many bytes of information can be transferred in single I/O operations. When used in conjunction with an interrupt, the CPU is notified only after the entire block of data has been transferred. For each byte or word transferred, it must provide the memory address and all the bus signals that control the data transfer.
- Interaction with a device controller is managed through a device driver.
- Device drivers are part of the operating system, but not necessarily part of the OS kernel. The operating system provides a simplified view of the device to user applications (e.g., character devices vs. block devices in UNIX). In some operating systems (e.g., Linux), devices are also accessible through the /dev file system.
- In some cases, the operating system buffers data that are transferred between a device and a user space program (disk cache, network buffer). This usually increases performance, but not always.
- Handshaking between the DMA controller and the device controller is performed via a pair of wires called DMA-request and DMA-acknowledge.
- The device controller places a signal on the DMA-request wire when a word of data is available for transfer.

- This signal causes the DMA controller to seize the memory bus, place the desired address on the memory-address wires, and place a signal on the DMA-acknowledge wire.

- When the device controller receives the DMA-acknowledge signal, it transfers the word of data to memory and removes the DMA-request signal.

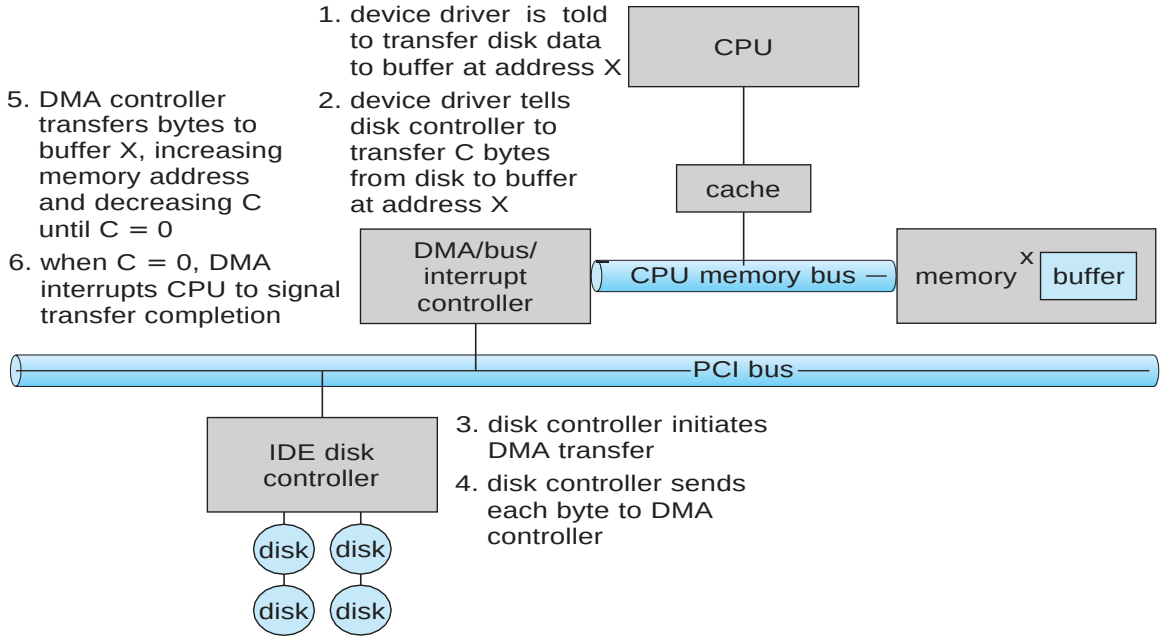


Figure 95: Six Step Process to Perform DMA Transfer

4.6. Application I/O Interfaced

- Structuring techniques and interfaces for the operating system enable I/O devices to be treated in a standard, uniform way. For instance, how an application can open a file on a disk without knowing what kind of disk it is, and how new disks and other devices can be added to a computer without the operating system being disrupted.
- The actual differences are encapsulated in ken modules called device drivers mat internally are custom tailored to each device but that export one of the standard interfaces.
- The purpose of the device-driver layer is to hide the differences among device controllers from the I/O subsystem of the kernel, much as the I/O system calls.
- **Character-stream or block**: A character-stream device transfers bytes one by one, whereas a block device transfers a block of bytes as a unit. Commands include read, write, and seek, Raw **I/O**, **direct I/O**, or file-system access.
    - Memory-mapped file access possible
    - File mapped to virtual memory and clusters brought via demand paging
    - DMA
    - Character devices include keyboards, mice, serial ports
    - Commands include **get()**, **put()**
    - Libraries layered on top allow line editing
- **Network Devices:** Varying enough from block and character to have own interface
    - Linux, Unix, Windows and many others include socket interface
    - Separates network protocol from network operation
    - Includes select() functionality
    - Approaches vary widely (pipes, FIFOs, streams, queues, mailboxes)

- **Sequential or random-access**: A sequential device transfers data in a fixed order that is determined by the device, whereas the user of a random-access device can instruct the device to seek to any of the available data storage locations.
- **Synchronous or asynchronous**: A synchronous device is one that performs data transfers with predictable response times. An asynchronous device exhibits irregular or unpredictable response times.
- **Sharable or dedicated**: A sharable device can be used concurrently by several processes or threads; a dedicated device cannot.
- **Speed of operation**: Device speeds range from a few bytes per second to a few gigabytes per second.
- **Read-write, read only, or write only**: Some devices perform both input and output, but others support only one data direction. For the purpose of application access, many of these differences are hidden by the operating system, and the devices are grouped into a few conventional types.
- Operating systems also provide special system calls to access a few additional devices, such as a time-of-day clock and a timer. The performance and addressing characteristics of network I/O differ significantly from those of disk I/O, most operating systems provide a network I/O interface that is different from the read-write-seek interface used for disks.

aspect	variation	example
data-transfer mode	character block	terminal disk
access method	sequential random	modem CD-ROM
transfer schedule	synchronous asynchronous	tape keyboard
sharing	dedicated sharable	tape keyboard
device speed	latency seek time transfer rate delay between operations	
I/O direction	read only write only read–write	CD-ROM graphics controller disk

Fig: Characteristic of I/O devices

4.6.1. Clocks and Timers

Most computers have hardware clocks and timers that provide three basic functions:

1. Give the current time
2. Give the elapsed time
3. Set a timer to trigger operation X at time T

These functions are used heavily by the operating system, and also by time sensitive applications. The hardware to measure elapsed time and to trigger operations is called a programmable interval timer.

- Provide current time, elapsed time, timer
- Normal resolution about 1/60 second
- Some systems provide higher-resolution timers
- **Programmable interval timer** used for timings, periodic interrupts
- **ioctl()** (on UNIX) covers odd aspects of I/O such as clocks and timers

4.6.2. Blocking and Non-blocking I/O

- One remaining aspect of the system-call interface relates to the choice between blocking I/O and non-blocking (asynchronous) I/O. When an application calls a blocking system call, the execution of the application is suspended. The application is moved from the operating system's run queue to a wait queue.
- After the system call completes, the application is moved back to the run queue, where it is eligible to resume execution, at which time it will receive the values returned by the system call.
- Implemented via multi-threading
- Returns quickly with count of bytes read or written
- **select()** to find if data ready then **read()** or **write()** to transfer

Asynchronous - process runs while I/O executes

- Difficult to use
- I/O subsystem signals process when I/O completed

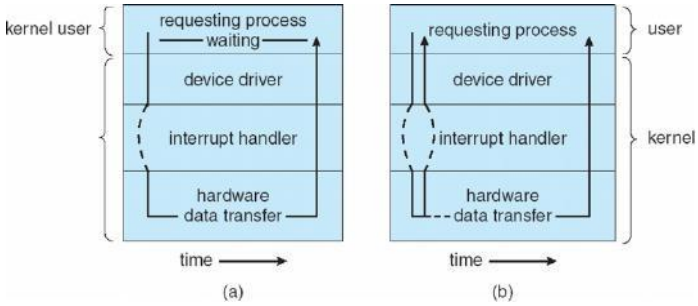


Figure 96: Two I/O model a) Synchronous b) Asynchronous

4.7. Kernel I/O Subsystem

- Kernels provide many services related to I/O. The services that we describe are I/O scheduling, buffering caching, spooling, device reservation, and error handling.

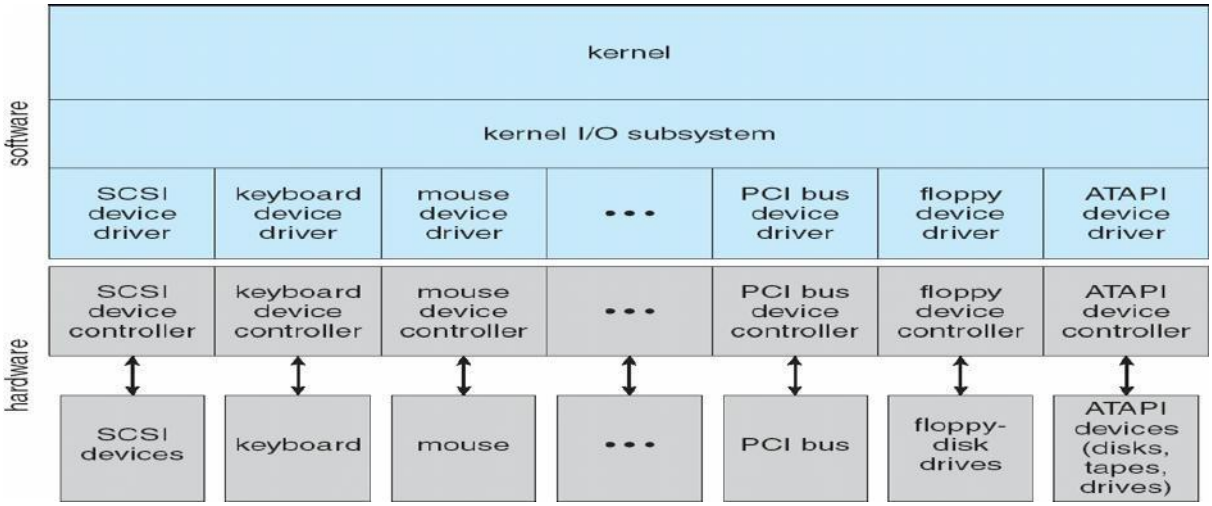


Figure 97: Kernel I/O Subsystem

4.7.1. Scheduling

- To schedule a set of I/O requests means to determine a good order in which to execute them. The order in which applications issue system calls rarely is the best choice. Scheduling can improve overall system performance, can share device access fairly among processes, and can reduce the average waiting time for I/O to complete. Operating-system developers implement scheduling by maintaining a queue of requests for each device. When an application issues a blocking I/O system call, the request is placed on the queue for that device.

- The I/O scheduler rearranges the order of the queue to improve the overall system efficiency and the average response time experienced by applications.

4.7.2. Buffering

- A buffer is a memory area that stores data while they are transferred between two devices or between a devices arid an application. Buffering is done for three reasons.
- One reason is to cope with a speed mismatch between the producer and consumer of a data stream.
- Second buffer while the first buffer is written to disk. A second use of buffering is to adapt between devices that have different data transfer sizes.
- A third use of buffering is to support copy semantics for application I/O.

4.7.3. Caching

- A cache is region of fast memory that holds copies of data. Access to the cached copy is more efficient than access to the original.
- Caching and buffering are two distinct functions, but sometimes a region of memory can be used for both purposes.

4.7.4. Spooling and Device Reservation

- A spool is a buffer that holds output for a device, such as a printer, that cannot accept interleaved data streams. The spooling system copies the queued spool files to the printer one at a time.
- In some operating systems, spooling is managed by a system daemon process. In other operating systems, it is handled by an in kernel thread.

4.7.5. Error Handling

- An operating system that uses protected memory can guard against many kinds of hardware and application errors.
- OS can recover from disk read, device unavailable, transient write failures
- Retry a read or write, for example
- Some systems more advanced – Solaris FMA, AIX
- Track error frequencies, stop using device with increasing frequency of retry-able errors
- Most return an error number or code when I/O request fails
- System error logs hold problem reports

4.7.6. Device drivers

- In computing, a device driver or software driver is a computer program allowing higher-level computer programs to interact with a hardware device.
- A driver typically communicates with the device through the computer bus or communications subsystem to which the hardware connects. When a calling program invokes a routine in the driver, the driver issues commands to the device. Once the device sends data back to the driver, the driver may invoke routines in the

original calling program. Drivers are hardware-dependent and operating-system-specific.

- They usually provide the interrupt handling required for any necessary asynchronous time-dependent hardware interface.

4.7.7. Vectored I/O

- Vectored I/O allows one system call to perform multiple I/O operations
- For example, Unix readve() accepts a vector of multiple buffers to read into or write from
- This scatter-gather method better than multiple individual I/O calls
- Decreases context switching and system call overhead
- Some versions provide atomicity
- Avoid for example worry about multiple threads changing data as reads / writes occurring

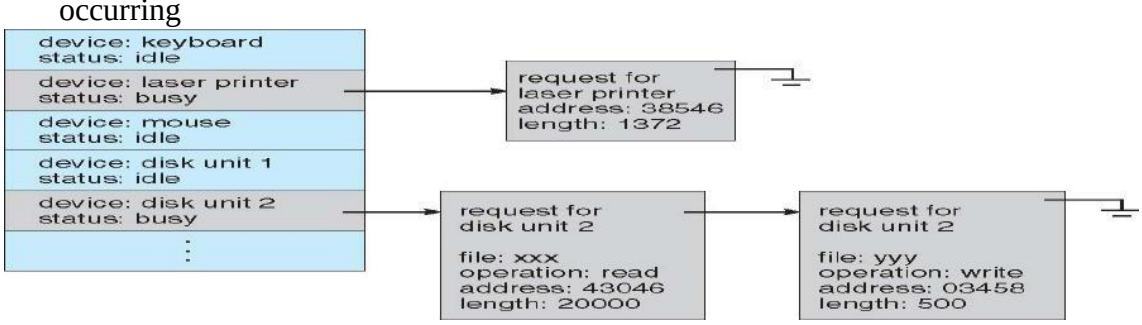


Figure 98: Device Status Table

4.7.8. I/O Protection

- User process may accidentally or purposefully attempt to disrupt normal operation via illegal I/O instructions
    - All I/O instructions defined to be privileged
        - I/O must be performed via system calls
        - Memory-mapped and I/O port memory locations must be protected too

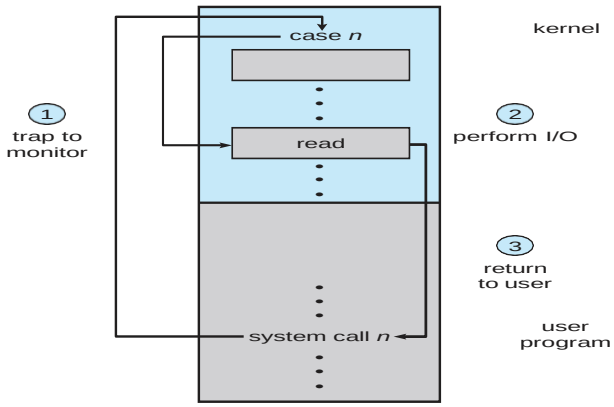


Figure 99: Use of a System Call to Perform I/O:

4.8. Kernel Data Structures

- Kernel keeps state info for I/O components, including open file tables, network connections, character device state
- Many, many complex data structures to track buffers, memory allocation, “dirty” blocks

- Some use object-oriented methods and message passing to implement I/O
- Windows uses message passing
- Message with I/O information passed from user mode into kernel
- Message modified as it flows through to device driver and back to process
- The message-passing approach can add overhead, by comparison with procedural techniques that use shared data structures, but it simplifies the structure and design of the I/O system and adds flexibility.

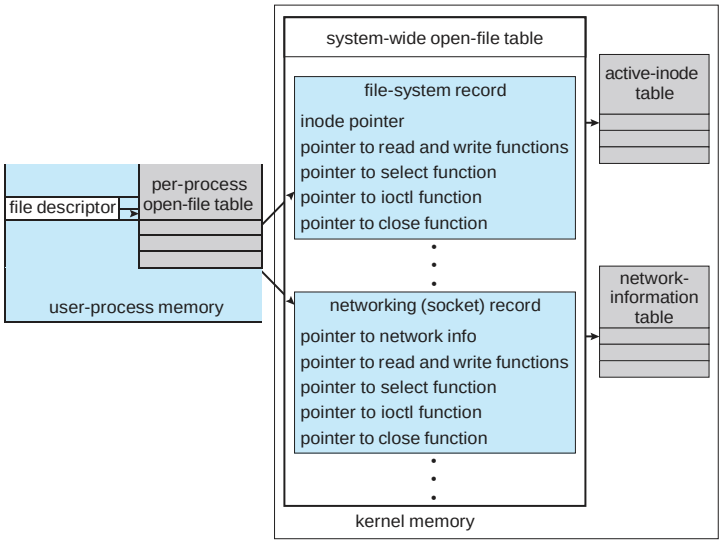


Figure 100: UNIX I/O Kernel Structure

4.9. Power Management

- Not strictly domain of I/O, but much is I/O related
- Computers and devices use electricity, generate heat, frequently require cooling
- OSes can help manage and improve use
  - Cloud computing environments move virtual machines between servers
    - Can end up evacuating whole systems and shutting them down
- Mobile computing has power management as first class OS aspect
- For example, Android implements
  - Component-level power management
    - Understands relationship between components
    - Build device tree representing physical device topology
    - System bus -> I/O subsystem -> {flash, USB storage}
    - Device driver tracks state of device, whether in use
    - Unused component – turn it off
    - All devices in tree branch unused – turn off branch
- Wake locks – like other locks but prevent sleep of device when lock is held
- Power collapse – put a device into very deep sleep
  - Marginal power use
  - Only awake enough to respond to external stimuli (button press, incoming call)

4.10 I/O Requests to Hardware Operations

- Consider reading a file from disk for a process:
  - Determine device holding file
  - Translate name to device representation
  - Physically read data from disk into buffer
  - Make data available to requesting process

- Return control to process

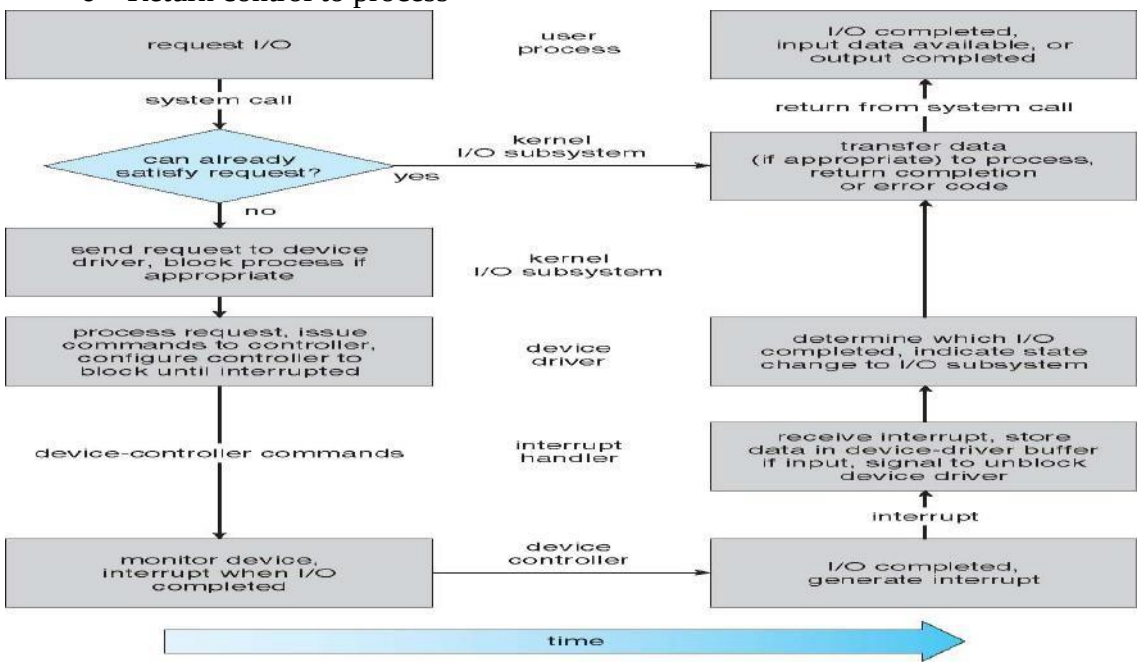


Figure 101: Life cycle of an I/O Request

4.10.1 Life Cycle of Blocking read request

1. A process issues a blocking read () system call to a file descriptor of a file that has been opened previously.
2. The system-call code in the kernel checks the parameters for correctness. In the case of input, if the data are already available in the buffer cache, the data are returned to the process, and the I/O request is completed.
3. Otherwise, a physical I/O must be performed. The process is removed from the run queue and is placed on the wait queue for the device, and the I/O request is scheduled. Eventually, the I/O subsystem sends the request to the device driver. Depending on the operating system, the request is sent via a subroutine call or an in-kernel message.
4. The device driver allocates kernel buffer space to receive the data and schedules the I/O. Eventually, the driver sends commands to the device controller by writing into the device-control register.
5. The device controller operates the device hardware to perform the data transfer.
6. The driver may poll for status and data, or it may have set up a DMA transfer into kernel memory.
7. The correct interrupt handler receives the interrupt via the interrupt vector table, stores any necessary data, signals the device driver, and returns from the interrupt.
8. The device driver receives the signal, determines which I/O request has completed, determines the request's status, and signals the kernel I/O subsystem that the request has been completed. ·
9. The kernel transfers data or return codes to the address space of the requesting process and moves the process from the wait queue back to the ready queue.

Performance

- I/O a major factor in system performance:
  - Demands CPU to execute device driver, kernel I/O code
  - Context switches due to interrupts

- Data copying
- Network traffic especially stressful

4.11 STREAMS

- STREAM – a full-duplex communication channel between a user-level process and a device in Unix System V and beyond
- A STREAM consists of:
  - STREAM head interfaces with the user process
  - driver end interfaces with the device
  - zero or more STREAM modules between them
- Each module contains a read queue and a write queue
- Message passing is used to communicate between queues
- Flow control option to indicate available or busy
- Asynchronous internally, synchronous where user process communicates with stream head

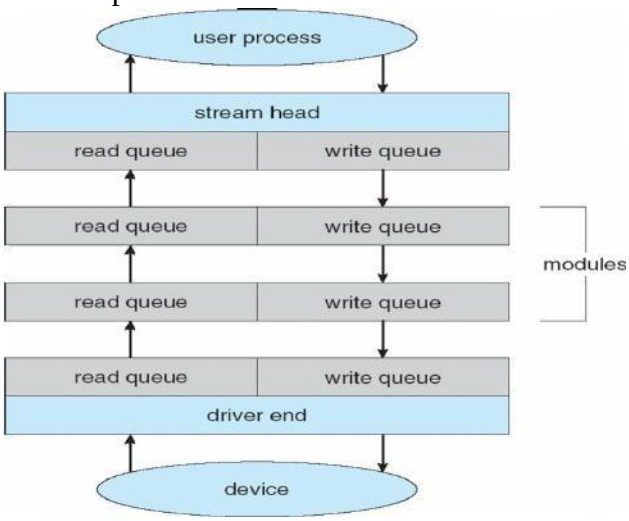


Figure 102: The STREAM Structure

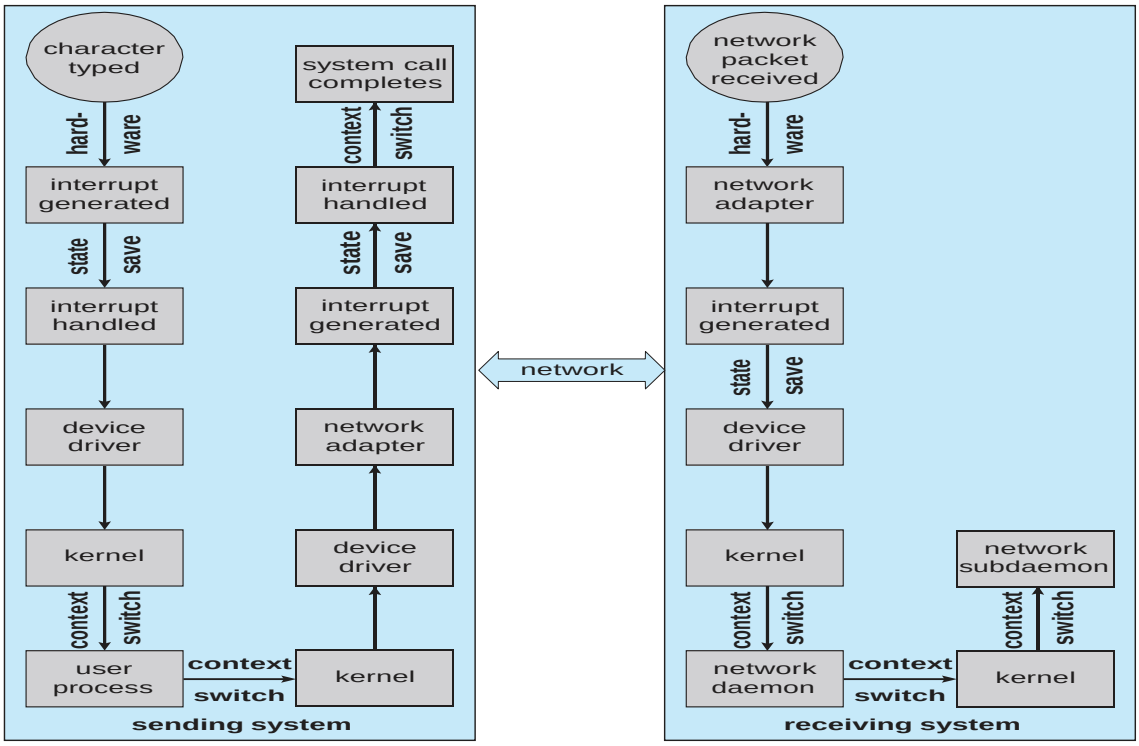


Figure 103: Intercomputer Communication

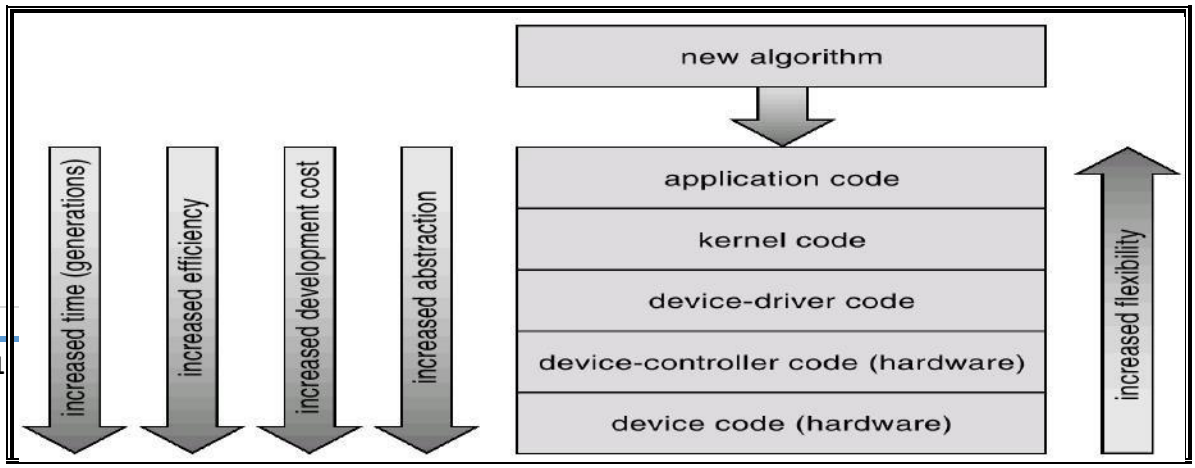
Improving Performance

Principles to improve the efficiency of I/O:

1. Reduce the number of context switches.
2. Reduce the number of times that data must be copied in memory while passing between device and application.
3. Reduce the frequency of interrupts by using large transfers, smart controllers, and polling (if busy waiting can be minimized).
4. Increase concurrency by using DMA-knowledgeable controllers or channels to offload simple data copying from the CPU.
5. Move processing primitives into hardware, to allow their operation in device controllers to be concurrent with CPU and bus operation.
6. Balance CPU, memory subsystem, bus, and I/O performance, because an overload in any one area will cause idleness in others.

4.12. Device Functionality

- Device driver is complex.
- It not only manages individual disks but also implements RAID arrays. To do so, it converts an application's read or write request into a coordinated set of disk I/O operations.
- It implements sophisticated error-handling and data-recovery algorithms and takes many steps to optimize disk performance.
- Where the I/O functionality should be implemented- in the device hardware, in the device driver, or in application software?
- Initially, we implement experimental I/O algorithms at the application level
    - because application code is flexible and application bugs are unlikely to cause system crashes.
    - Developing code at the application level, can avoid the need to reboot or reload device drivers after every change to the code.
- But, an application-level implementation can be inefficient
    - Because of the overhead of context switches and because the application cannot take advantage of internal kernel data structures and kernel functionality (threading, locking etc.)
- Application-level algorithm may re-implement it in the kernel.
    - This can improve performance, but the development effort is more challenging, because an operating-system kernel is a large, complex software system. Moreover, an in-kernel implementation must be thoroughly debugged to avoid data corruption and system crashes.
- The highest performance may be obtained through a specialized implementation in hardware, either in the device or in the controller.
    - The disadvantages of a hardware implementation include the difficulty and expense of making further improvements or of fixing bugs, the increased development time (months rather than days), and the decreased flexibility.



4.13. Mass-Storage Device

4.13.1. Disk Structure

- Disk provide bulk of secondary storage of computer system. The disk can be considered the one I/O device that is common to each and every computer. Disks come in many size and speeds, and information may be stored optically or magnetically. Magnetic tape was used as an early secondary storage medium, but the access time is much slower than for disks. For backup, tapes are currently used.
- Modern disk drives are addressed as large one dimensional arrays of logical blocks, where the logical block is the smallest unit of transfer. The actual details of disk I/O operation depends on the computer system, the operating system and the nature of the I/O channel and disk controller hardware.
- The basic unit of information storage is a sector. The sectors are stored on a flat, circular, media disk. This media spins close to one or more read/write heads. The heads can move from the inner portion of the disk to the outer portion.
- When the disk drive is operating, the disk is rotating at constant speed. To read or write, the head must be positioned at the desired track and at the beginning of the desired sector on that track. Track selection involves moving the head in a movable head system or electronically selecting one head on a fixed head system. These characteristics are common to floppy disks, hard disks, CD-ROM and DVD.
- Drives rotate at 60 to 200 times per second
- **Transfer rate** is rate at which data flow between drive and computer
- **Positioning time** (**random-access time**) is time to move disk arm to desired cylinder (**seek time**) and time for desired sector to rotate under the disk head (**rotational latency**)
- **Head crash** results from disk head making contact with the disk surface
- That's bad (no recovery, must replace disk)
- Disks can be removable
- Drive attached to computer via **I/O bus**
- Buses vary, including **EIDE, ATA, SATA, USB, Fibre Channel, SCSI**
- **Host controller** in computer uses bus to talk to **disk controller** built into drive or storage array

4.13.2. Disk Attachment

- Host-attached storage accessed through I/O ports talking to I/O buses
- SCSI itself is a bus, up to 16 devices on one cable, **SCSI initiator** requests operation and **SCSI targets** perform tasks
    - Each target can have up to 8 **logical units** (disks attached to device controller)
- Fiber Channel (FC) is high-speed serial architecture
    - Can be switched fabric with 24-bit address space – the basis of **storage area networks** (**SAN**s) in which many hosts attach to many storage units
    - Can be **arbitrated loop** (**FC-AL**) of 126 devices

4.13.3. Network-Attached Storage

- Network-attached storage (NAS) is storage made available over a network rather than over a local connection (such as a bus)

- Network File System (NFS) and Common Internet File System (CIFS) are common protocols
- Implemented via remote procedure calls (RPCs) between host and storage
- New iSCSI (latest network attached storage protocol) protocol uses IP network to carry the SCSI protocol

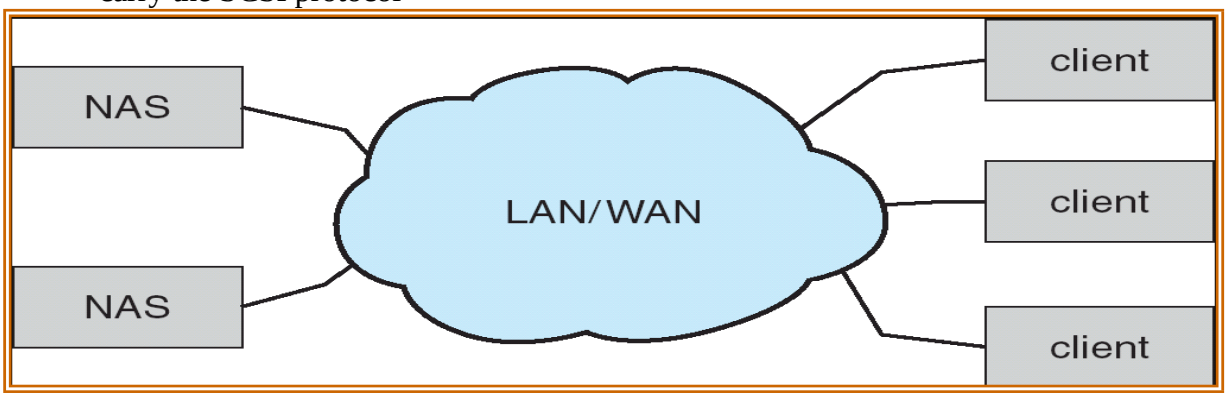


Figure 104: Networked Attached Storage

4.13.4. Storage Area Network

- One drawback of network-attached storage systems is that the storage I/O operations consume bandwidth on the data network, thereby increasing the latency of network communication.
- Storage Area Network (SAN) is a private network connecting server and storage unit.
- Common in large storage environments (and becoming more common)
- Multiple hosts attached to multiple storage arrays - flexible

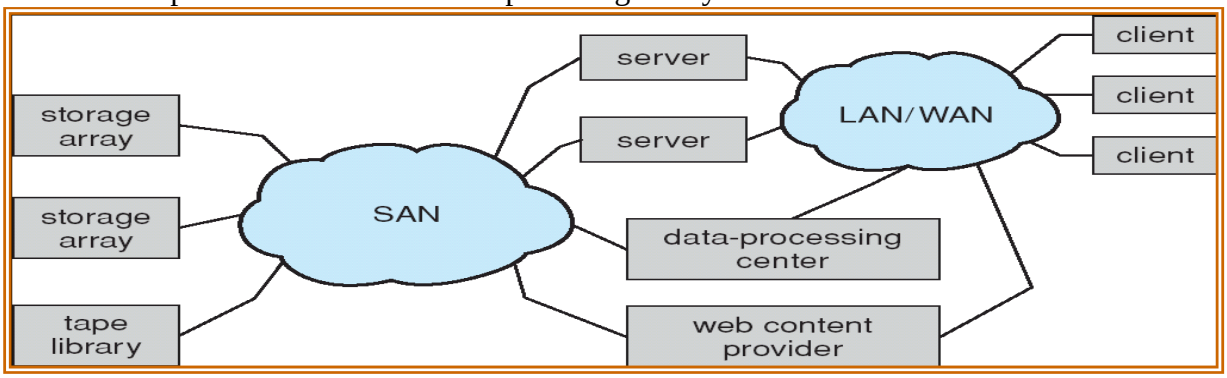


Figure 105: Storage Area Network

4.13.5. Disk Scheduling

- The operating system is responsible for using hardware efficiently for the disk drives, this means having a fast access time and disk bandwidth.
- Access time has two major components
  - ***Seek time*** is the time for the disk arc to move the heads to the cylinder containing the desired sector.
  - ***Rotational latency*** is the additional time waiting for the disk to rotate the desired sector to the disk head.
  - Minimize seek time
  - Seek time ≈ seek distance
- Disk bandwidth is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer.

- The amount of head needed to satisfy a series of I/O request can affect the performance. If desired disk drive and controller are available, the request can be serviced immediately. If a device or controller is busy, any new requests for service will be placed on the queue of pending requests for that drive. When one request is completed, the operating system chooses which pending request to service next.

Different types of scheduling algorithms are as follows.

1. First Come, First Served scheduling algorithm (FCFS).
2. Shortest Seek Time First (SSTF) algorithm
3. SCAN algorithm
4. Circular SCAN (C-SCAN) algorithm
5. LOOK Scheduling Algorithm
6. C-LOOK

4.13.5.1. First Come, First Served scheduling algorithm (FCFS).

- The simplest form of scheduling is first-in-first-out (FIFO) scheduling, which processes items from the queue in sequential order. This strategy has the advantage of being fair, because every request is honored and the requests are honored in the order received. With FIFO, if there are only a few processes that require access and if many of the requests are to clustered file sectors, then we can hope for good performance.
- Priority With a system based on priority (PRI), the control of the scheduling is outside the control of disk management software.
- Last In First Out ln transaction processing systems, giving the device to the most recent user should result. In little or no arm movement for moving through a sequential file. Taking advantage of this locality improves throughput and reduces queue length.

Illustration of several algorithm with a request queue (0-199).
98, 183, 37, 122, 14, 124, 65, 67
Head pointer 53

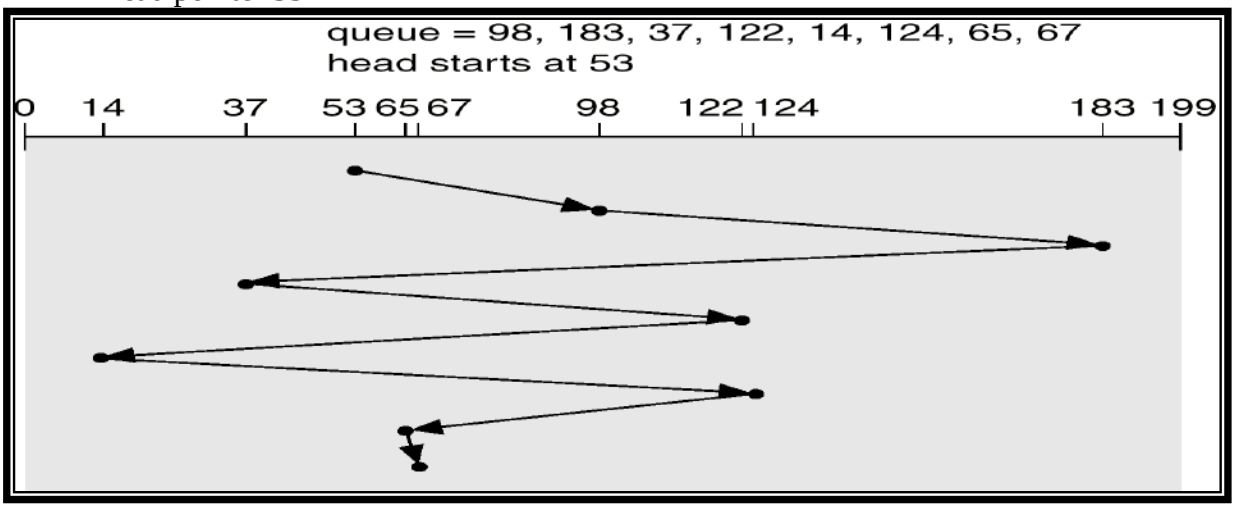


Figure 106: FIFO disk scheduling algorithm

Total head movement = 640 cylinders.

4.13.5.2. Shortest Seek Time First (SSTF) algorithm

- The SSTF policy is to select the disk I/O request the requires the least movement of the disk arm from its current position. Scan With the exception of FIFO, all of the policies described so far can leave some request unfulfilled until the entire queue is emptied. That is, there may always be new requests arriving that will be chosen before an existing request.
- The choice should provide better performance than FCFS algorithm.
- Under heavy load, SSTF can prevent distant request from ever being serviced.
- This phenomenon is known as starvation. SSTF scheduling is essentially a form of shortest job first scheduling.
- SSTF scheduling algorithm are not very popular because of two reasons.
    - Starvation possibly exists.
    - It increases higher overheads.

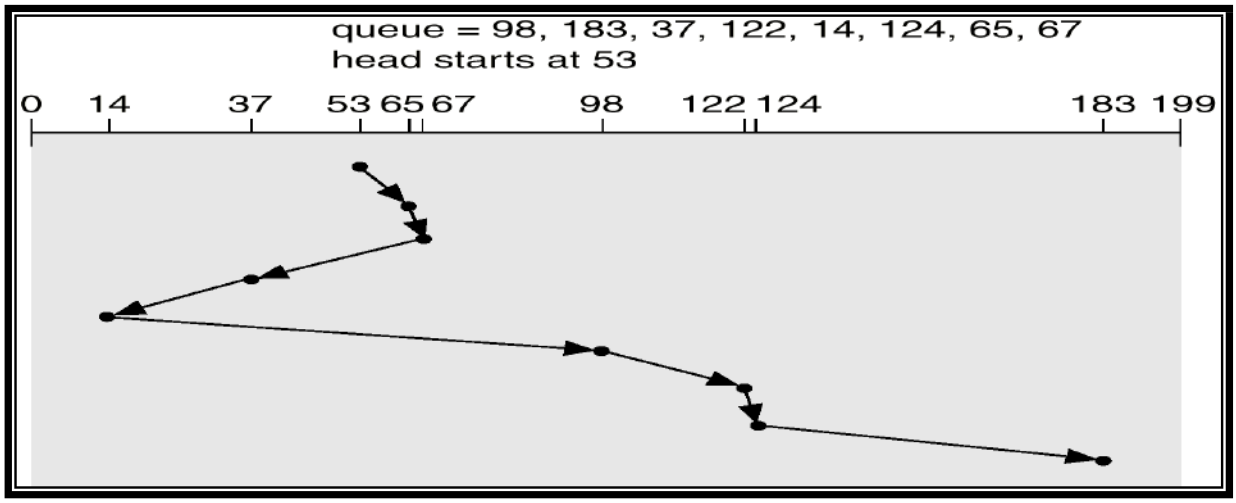


Figure 107: SSTF disk scheduling Algorithm

Total head movement = 236 cylinders.

4.13.5.3. SCAN scheduling algorithm

- The scan algorithm has the head start at track 0 and move towards the highest numbered track, servicing all requests for a track as it passes the track. The service direction is then reserved and the scan proceeds in the opposite direction, again picking up all requests in order.
- SCAN algorithm is guaranteed to service every request in one complete pass through the disk. SCAN algorithm behaves almost identically with the SSTF algorithm. The SCAN algorithm is sometimes called ***elevator*** algorithm.

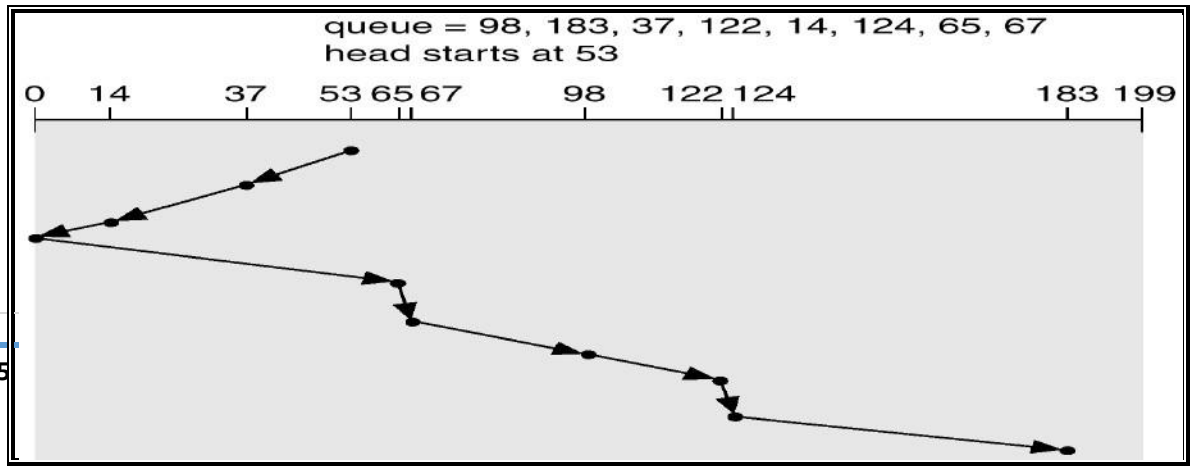


Figure 108: SCAN disk scheduling algorithm

Total head movement = 236 cylinders.

4.13.5.4. C-SCAN Scheduling Algorithm

- The C-SCAN policy restricts scanning to one direction only. Thus, when the last track has been visited in one direction, the arm is returned to the opposite end of the disk and the scan begins again.
- This reduces the maximum delay experienced by new requests.

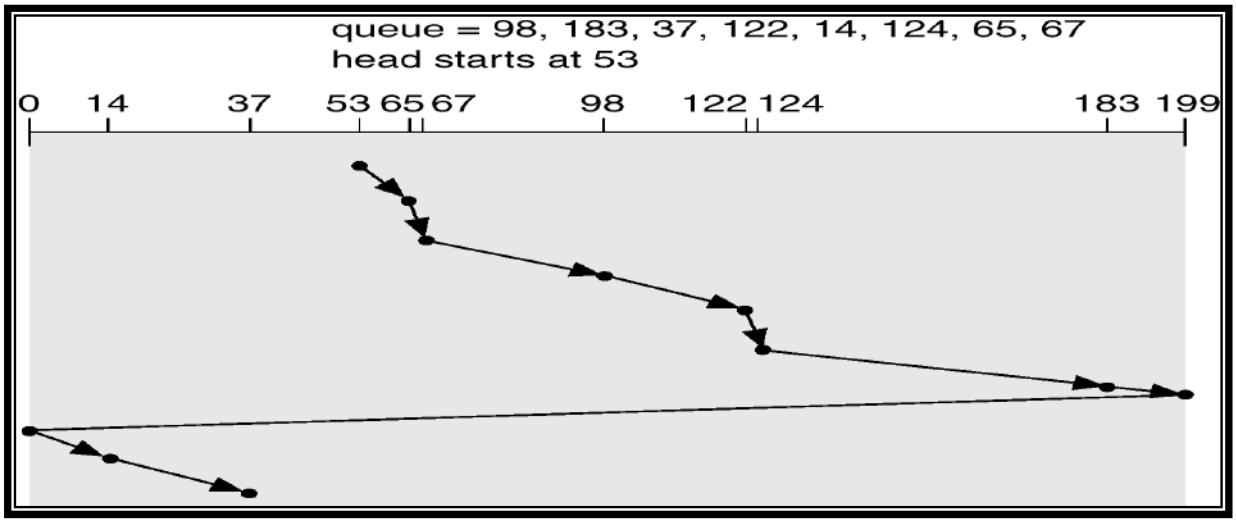


Figure 109: C-SCAN disk scheduling algorithm

4.13.5.5. LOOK Scheduling Algorithm

- Start the head moving in one direction. Satisfy the request for the closest track in that direction when there is no more request in the direction, the head is traveling, reverse direction and repeat. This algorithm is similar to innermost and outermost track on each circuit.

4.13.5.6. C-LOOK Scheduling Algorithm

- Version of C-SCAN
- Arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk.

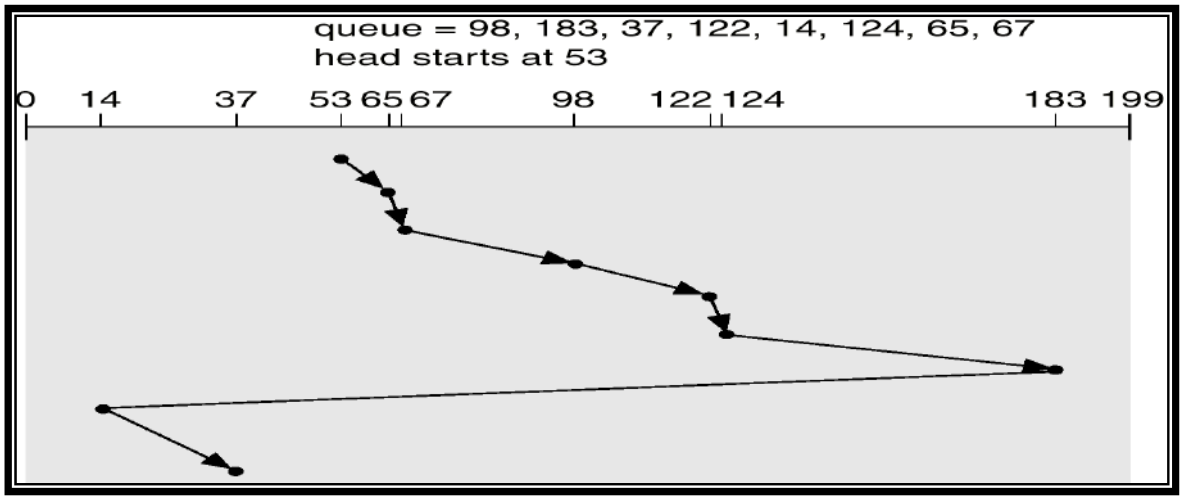


Figure 110: C-LOOK disk scheduling Algorithm

4.13.5.7. Selecting a Disk-Scheduling Algorithm

- SSTF is common and has a natural appeal.
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk.
- Performance depends on the number and types of requests.
- Requests for disk service can be influenced by the file-allocation method.
- The disk-scheduling algorithm should be written as a separate module of the operating system, allowing it to be replaced with a different algorithm if necessary.
- Either SSTF or LOOK is a reasonable choice for the default algorithm.

4.13.6. Disk Management

Operating system is responsible for disk management. Following are some activities discussed.

4.13.6.1. Disk Formatting

Disk formatting is of two types.

a. Physical formatting or low level formatting.
b. Logical Formatting

Physical Formatting

- Disk must be formatted before storing data.
- Disk must be divided into sectors that the disk controllers can read/write.
- Low level formatting files the disk with a special data structure for each sector.
- Data structure consists of three fields: *header*, *data area* and *trailer*.
- Header and trailer contain information used by the disk controller.
- Sector number and Error Correcting Codes (ECC) contained in the header and trailer.
- For writing data to the sector – ECC is updated.
- For reading data from the sector – ECC is recalculated.
- If the stored and calculated numbers are different, this mismatch indicates that the data area of the sector has become corrupted and that the disk sector may be bad (**Bad Sector**).
- Low level formatting is done at factory.

Logical Formatting

- After disk is partitioned, logical formatting used.
- Operating system stores the initial file system data structures onto the disk.

4.13.6.2. Boot Block

- When a computer system is powered up or rebooted, a program in read only memory executes.
- Diagnostic check is done first.
- Stage 0 boot program is executed.
- Boot program reads the first sector from the boot device and contains a stage-1 boot program.
- May be boot sector will not contain a boot program.

- PC booting from hard disk, the boot sector also contains a partition table.
- The code in the boot ROM instructs the disk controller to read the boot blocks into memory and then starts executing that code.
- Full boot strap program is more sophisticated than the bootstrap loader in the boot ROM.
- Methods such as *sector sparing* used to handle bad blocks.

4.13.6.3. Bad Blocks

- Because disks have moving parts and small tolerances, they are prone to failure.
- The disk needs to be replaced and its contents restored from backup media to the new disk.
- More frequently, one or more sectors become defective. Most disks even come from the factory with bad blocks.
- Depending on the disk and controller in use, these blocks are ***handled in a variety of ways.***

1. Manually

- On simple disks, such as some disks with IDE controllers, bad blocks are handled manually.
- Example: In MS-DOS, format command performs logical formatting and, as a part of the process, scans the disk to find bad blocks.
- If format finds a bad block, it writes a special value into the corresponding FAT entry to tell the allocation routines not to use that block.
- If blocks go bad during normal operation, a special program (such as *chkdsk*) must be run manually to search for the bad blocks and to lock them away.
- Data that resided on the bad blocks usually are lost.

2. Sector Sparing or Forwarding

- In more sophisticated disks, such as the SCSI disks used in high-end PCs and most workstations and servers, are smarter about bad-block recovery.
- The controller maintains a list of bad blocks on the disk. The list is initialized during the low-level formatting at the factory and is updated over the life of the disk.
- Low-level formatting also sets aside spare sectors not visible to the operating system.
- The controller can be told to replace each bad sector logically with one of the spare sectors. This scheme is known as sector sparing or forwarding.
- A typical bad-sector transaction might be as follows:
  - The operating system tries to read logical block 7.
  - The controller calculates the ECC and finds that the sector is bad. It reports this finding to the operating system.
  - The next time the system is rebooted, a special command is run to tell the SCSI controller to replace the bad sector with a spare.
  - After that, whenever the system requests logical block 7, the request is translated into the replacement sector's address by the controller.
  - Most disks are formatted to provide a few spare sectors in each cylinder and a spare cylinder as well. When a bad block is remapped, the controller uses a spare sector from the same cylinder, if possible.

3. Sector Slipping

- Some controllers can be instructed to replace a bad block by *sector slipping*.

- Example: Suppose that logical block 7 becomes defective and the first available spare follows sector 30 and 31.
- Then, sector slipping remaps all the sectors front 7 to 30, moving them all down one spot. That is, sector 30 is copied into the spare, then sector 29 into 30, then 28 into 29, and so on, until sector 7 is copied into sector 8.
- Slipping the sectors in this way frees up the space of sector 8, so sector 7 can be mapped to it.
- The replacement of a bad block generally is not totally automatic because the data in the bad block are usually lost.

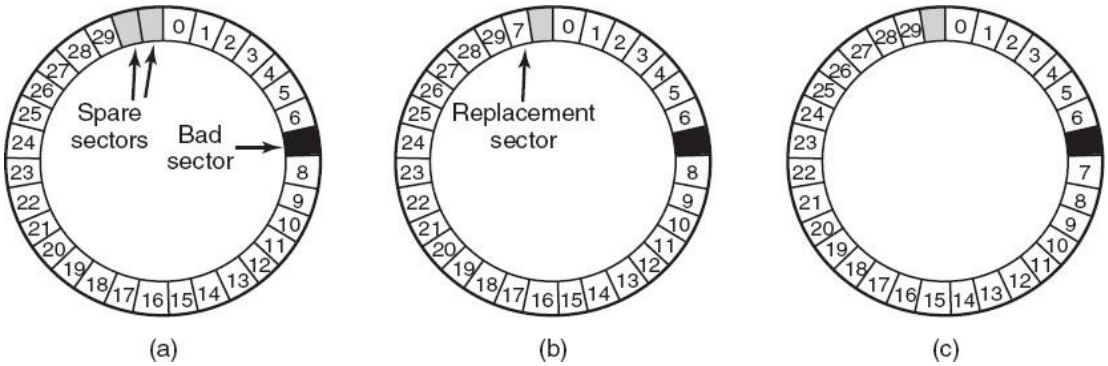


Figure 111: Bad Block Recovery (a) A disk track with a bad sector. (Sector 7 is bad) (b) Substituting a spare for the bad sector. (Sector Sparing or Forwarding (c) Shifting all the sectors to bypass the bad one. (Sector Slipping)

4.13.6.4. Interleaving

- Interleaving is a process or methodology to make a system more efficient, fast and reliable by arranging data in a non-contiguous manner. There are many uses for interleaving at the system level, including:
- Storage: As hard disks and other storage devices are used to store user and system data, there is always a need to arrange the stored data in an appropriate way.
- Interleaving is also known as sector interleave.

Why Interleaving?

- Reading continuously at very high rate requires a large buffer in the controller. **Example**: a controller with a one-sector buffer that has been given a command to read two consecutive sectors. After reading the first sector from the disk and doing the ECC calculation, the data must be transferred to main memory. While this transfer is taking place, the next sector will fly by the head. When the copy to memory is complete, the controller will have to wait almost an entire rotation time for the second sector to come around again.

Interleaved sectors

Fig: Copying to a buffer takes time; could wait a disk rotation before head reads next sector. So interleave sectors to avoid this fig (b, c)

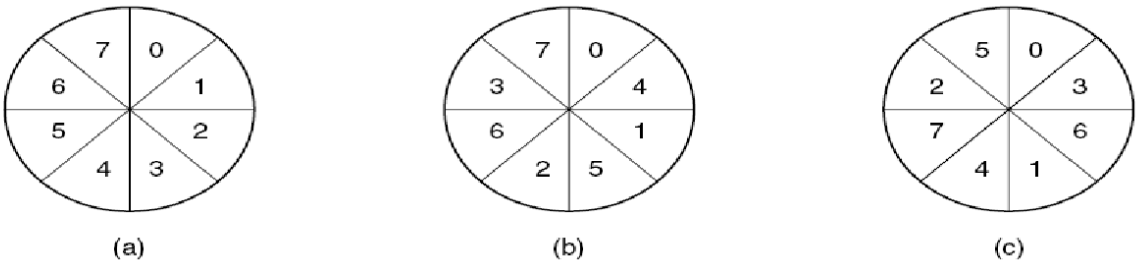


Figure 112: Interleaving (a) No Interleaving (b) Single Interleaving (c) Double Interleaving

4.13.7. Swap Space Management

Swap space management is low level task of the operating system. The main goal for the design and implementation of swap space is to provide the best throughput for the virtual memory system.

4.13.7.1.Swap-Space Use

The operating system needs to release sufficient main memory to bring in a process that is ready to execute. Operating system uses this swap space in various way. Paging systems may simply store pages that have been pushed out of main memory. Unix operating system allows the use of multiple swap space are usually put on separate disks, so the load placed on the I/O system by paging and swapping can be spread over the systems I/O devices.

4.13.7.2.Swap Space Location

Swap space can reside in two places:

1. Separate disk partition
2. Normal file System

- If the swap space is simply a large file within the file system, normal file system routines can be used to create it, name it and allocate its space. This is easy to implement but also inefficient. External fragmentation can greatly increase swapping times. Catching is used to improve the system performance. Block of information is cached in the physical memory, and by using special tools to allocate physically continuous blocks for the swap file.
- Swap space can be created in a separate disk partition. No file system or directory structure is placed on this space. A separate swap space storage manager is used to allocate and deallocate the blocks. This manager uses algorithms optimized for speed. Internal fragmentation may increase. Some operating systems are flexible and can swap both in raw partitions and in file system space.

4.13.8. Stable Storage Implementation

- The write ahead log, which required the availability of stable storage.
- By definition, information residing in stable storage is never lost.
- To implement such storage, we need to replicate the required information on multiple storage devices (usually disks) with independent failure modes.
- We also need to coordinate the writing of updates in a way that guarantees that a failure during an update will not leave all the copies in a damaged state and that, when we are recovering from failure, we can force all copies to a consistent and correct value, even if another failure occurs during the recovery.

4.13.8.1.Tertiary Storage Devices

- Low cost is the defining characteristic of tertiary storage.
- Generally, tertiary storage is built using *removable media*
- Common examples of removable media are floppy disks and CD-ROMs; other types are available.

4.13.8.2.Removable Disks

- Floppy disk — thin flexible disk coated with magnetic material, enclosed in a protective plastic case.
    - Most floppies hold about 1 MB; similar technology is used for removable disks that hold more than 1 GB.
    - Removable magnetic disks can be nearly as fast as hard disks, but they are at a greater risk of damage from exposure.
- A magneto-optic disk records data on a rigid platter coated with magnetic material.
    - Laser heat is used to amplify a large, weak magnetic field to record a bit.
    - Laser light is also used to read data (Kerr effect).
    - The magneto-optic head flies much farther from the disk surface than a magnetic disk head, and the magnetic material is covered with a protective layer of plastic or glass; resistant to head crashes.
- Optical disks do not use magnetism; they employ special materials that are altered by laser light.

4.13.8.3.WORM Disks

- The data on read-write disks can be modified over and over.
- WORM ("Write Once, Read Many Times") disks can be written only once.
- Thin aluminum film sandwiched between two glass or plastic platters.
- To write a bit, the drive uses a laser light to burn a small hole through the aluminum; information can be destroyed by not altered.
- Very durable and reliable.
- *Read Only* disks, such as CD-ROM and DVD, come from the factory with the data pre-recorded.

4.13.8.4.Tapes

- Compared to a disk, a tape is less expensive and holds more data, but random access is much slower.
- Tape is an economical medium for purposes that do not require fast random access, e.g., backup copies of disk data, holding huge volumes of data.
- Large tape installations typically use robotic tape changers that move tapes between tape drives and storage slots in a tape library.
    - stacker – library that holds a few tapes
    - silo – library that holds thousands of tapes
- A disk-resident file can be *archived* to tape for low cost storage; the computer can *stage* it back into disk storage for active use.

4.13.8.5.Disk Reliability

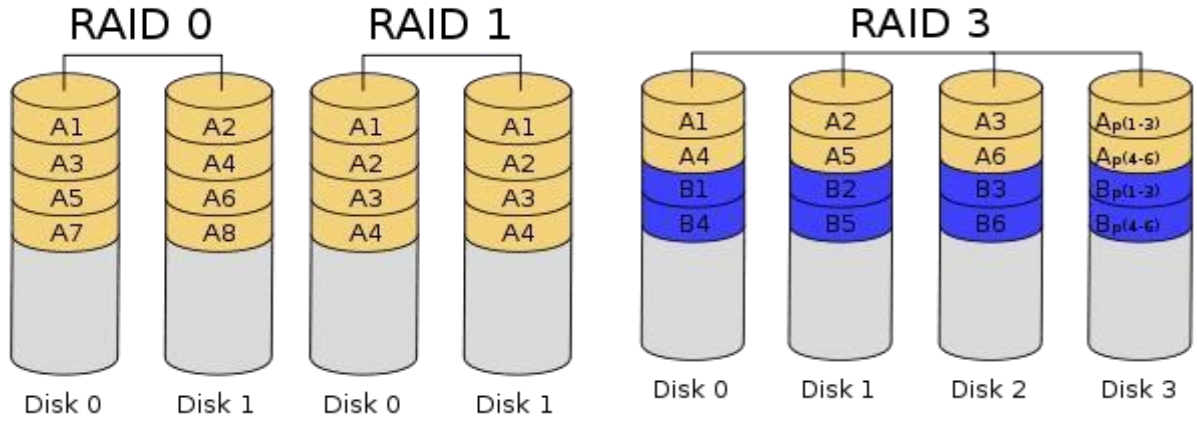
- Good performance means high speed, another important aspect of performance is reliability.
- A fixed disk drive is likely to be more reliable than a removable disk or tape drive.
- An optical cartridge is likely to be more reliable than a magnetic disk or tape.
- A head crash in a fixed hard disk generally destroys the data, whereas the failure of a tape drive or optical disk drive often leaves the data cartridge unharmed.

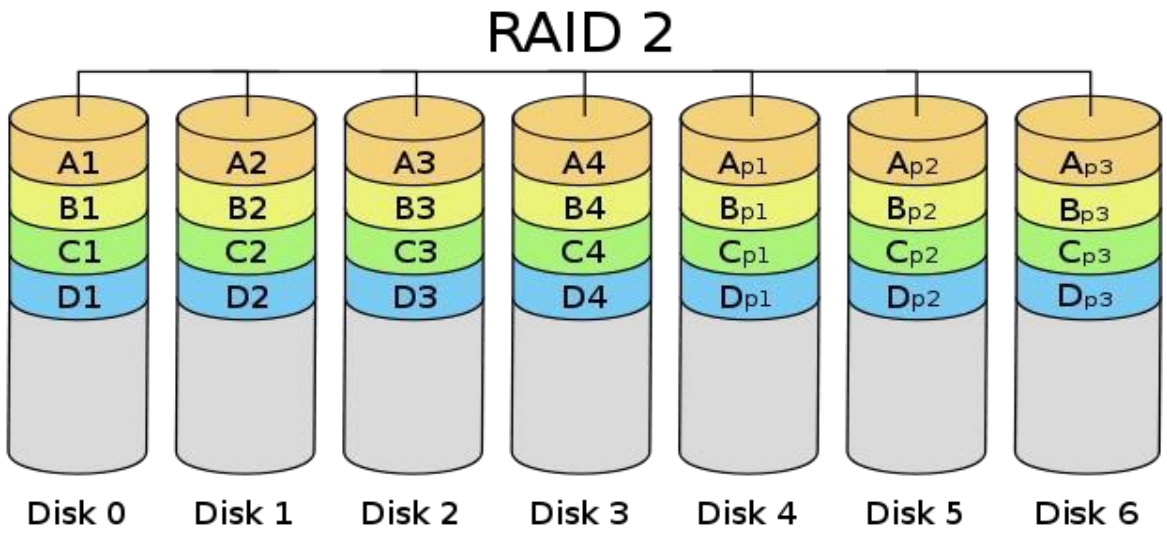
4.13.8.6.RAID (Redundant Array of Independent Disk)

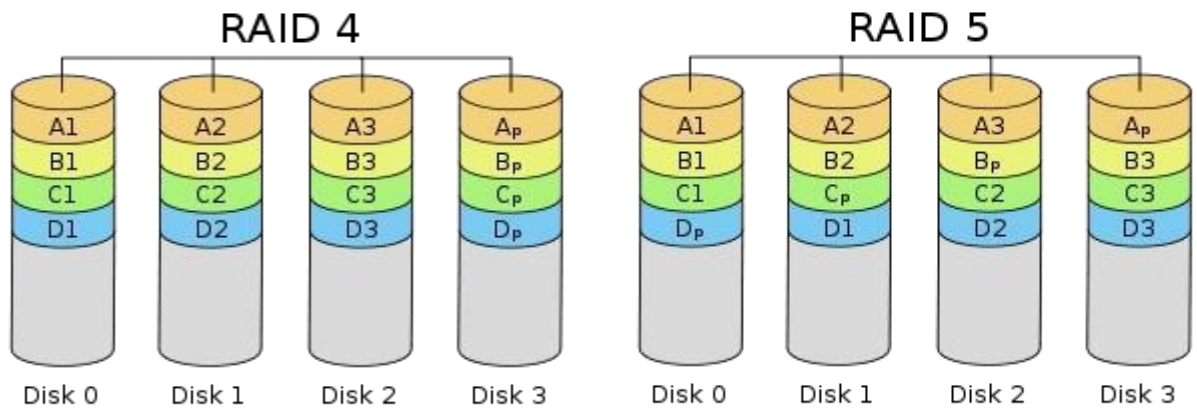
RAID (redundant array of independent disks), originally redundant array of inexpensive disks is a storage technology that combines multiple disk drive components into a logical unit. Data is distributed across the drives in one of several ways called "RAID levels", depending on what level of redundancy and performance (via parallel communication) is required.

RAID Levels

RAID 0 (block-level striping without parity or mirroring) has no (or zero) redundancy. It provides improved performance and additional storage but no fault tolerance. Hence simple stripe sets are normally referred to as RAID 0. Any drive failure destroys the array, and the likelihood of failure increases with more drives in the array (at a minimum, catastrophic data loss is almost twice as likely compared to single drives without RAID). A single drive failure destroys the entire array because when data is written to a RAID 0 volume, the data is broken into fragments called blocks. The number of blocks is dictated by the stripe size, which is a configuration parameter of the array. The blocks are written to their respective drives simultaneously on the same sector. This allows smaller sections of the entire chunk of data to be read off each drive in parallel, increasing bandwidth. RAID 0 does not implement error checking, so any error is uncorrectable. More drives in the array means higher bandwidth, but greater risk of data loss







In **RAID 1** (mirroring without parity or striping), data is written identically to multiple drives, thereby producing a "mirrored set"; at least 2 drives are required to constitute such an array. While more constituent drives may be employed, many implementations deal with a maximum of only 2; of course, it might be possible to use such a limited level 1 RAID itself as a constituent of a level 1 RAID, effectively masking the limitation. The array continues to operate as long as at least one drive is functioning. With appropriate operating system support, there can be increased read performance, and only a minimal write performance reduction; implementing RAID 1 with a separate controller for each drive in order to perform simultaneous reads (and writes) is sometimes called multiplexing (or duplexing when there are only 2 drives).

In **RAID 2** (bit-level striping with dedicated Hamming-code parity), all disk spindle rotation is synchronized, and data is striped such that each sequential bit is on a different drive. Hamming-code parity is calculated across corresponding bits and stored on at least one parity drive.

In **RAID 3** (byte-level striping with dedicated parity), all disk spindle rotation is synchronized, and data is striped so each sequential byte is on a different drive. Parity is calculated across corresponding bytes and stored on a dedicated parity drive.

RAID 4 (block-level striping with dedicated parity) is identical to RAID 5 (see below), but confines all parity data to a single drive. In this setup, files may be distributed between multiple drives. Each drive operates independently, allowing I/O requests to be performed in parallel. However, the use of a dedicated parity drive could create a performance bottleneck; because the parity data must be written to a single, dedicated parity drive for each block of non-parity data, the overall write performance may depend a great deal on the performance of this parity drive.

RAJD 5 (block-level striping with distributed parity) distributes parity along with the data and requires all drives but one to be present to operate; the array is not destroyed by a single drive failure. Upon drive failure, any subsequent reads can be calculated from the distributed parity such that the drive failure is masked from the end user. However, a single drive failure results in reduced performance of the entire array until the failed drive has been replaced and the associated data rebuilt. Additionally, there is the potentially disastrous RAID 5 write hole. RAID 5 requires at least 3 disks.

RAID 6 (block-level striping with double distributed parity) provides fault tolerance of two drive failures; the array continues to operate with up to two failed drives. This makes larger

RAID groups more practical, especially for high-availability systems. This becomes increasingly important as large- capacity drives lengthen the time needed to recover from the failure of a single drive. Single-parity RAID levels are as vulnerable to data loss as a RAID 0 array until the failed drive is replaced and its data rebuilt; the larger the drive, the longer the rebuild takes. Double parity gives additional time to rebuild the array without the data being at risk if a single additional drive fails before the rebuild is complete.

Summary

- The basic hardware elements involved in I/O buses, device controllers, and the device themselves. The work of moving data between devices and main memory is performed by the CPU as programmed I/O or is offloaded to a DMA controller. The kernel module that controls a device driver.
- The system call interface provided to applications is designed to handle several basic categories of hardware, including block devices, character devices, memory mapped files, network sockets, and programmed interval timers. The system calls usually block the processes that issue them, but non-blocking and asynchronous calls are used by the kernel itself and by applications that must not sleep while waiting for an I/O operation to complete.
- The kernel‘s I/O subsystem provides numerous services. Among these are I/O scheduling, buffering, caching, spooling, device reservation, and error handling.
- The system call interface provided to applications is designed to handle several basic categories of hardware, including block devices, character devices, memory mapped files, network sockets, and programmed interval timers.
- The system calls usually block the processes that issue them, but non-blocking and asynchronous calls are used by the kernel itself and by applications that must not sleep while waiting for an I/O operation to complete.
- I/O system calls are costly in terms of CPU consumption because of the many layers of software between a physical device and an application.
- Disk drives are the major secondary storage I/O devices on most computers. Most secondary devices are either magnetic disks or magnetic tapes. Modern disk drives are structured as large one dimensional arrays of logical disk blocks. Disk scheduling algorithms can improve the effective bandwidth, the average response time, and the variance response time. Algorithms such as SSTF, SCAN, C-SCAN, LOOK, and CLOOK are designed to make such improvements through strategies for disk queue ordering.
- Performance can be harmed by external fragmentation. The operating system manages block. First, a disk must be low level formatted to create the sectors on the raw hardware, new disks usually come preformatted. Then, disk is partitioned, file systems are created, and boot blocks are allocated to store the system bootstrap program. Finally when a block is corrupted, the system must have a way to lock out that block or to replace it logically with a space.
- Because of efficient swap space is a key to good performance, systems usually bypass the file system and use raw disk access for paging I/O. Some systems dedicate a raw disk partition to swap space, and others use a file within the file system instead.
- The write-ahead log scheme requires the availability of stable storage. Tertiary storage is built from disk and tape drives that use removable media. Many different technologies are available, including magnetic tape, removable magnetic and magneto-optic disks, and optical disks. Three important aspects of performance are bandwidth, latency, and reliability. Many bandwidths are available for both disks and tapes, but the random-access latency for a tape is generally much greater than that for a disk.